

Fundamentos e Introducción al Aprendizaje Automático

Material del curso

Universidad de Salamanca

Facultades de Ciencias y Ciencias Químicas

10 de septiembre de 2026

① Fundamentos

- Historia y evolución del aprendizaje automático
- Conceptos fundamentales

② Aprendizaje Supervisado

- Regresión
- Clasificación

③ Aprendizaje No Supervisado

- Clustering
- Reducción de dimensionalidad
- Detección de anomalías

Sección 1

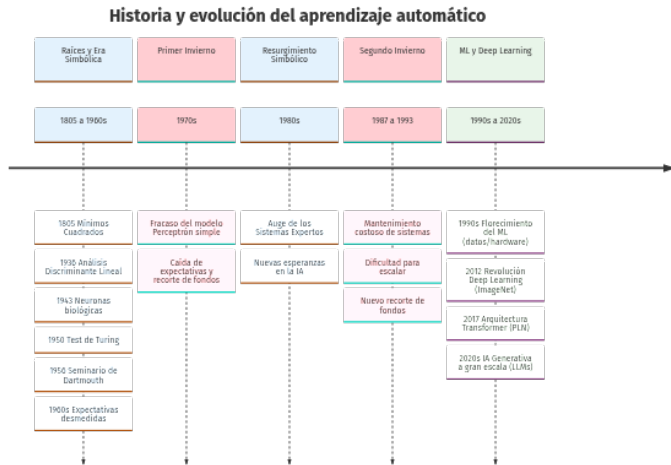
Fundamentos

1.1 Historia y evolución del ML

- El aprendizaje automático representa un **nuevo paradigma de programación**
- En vez de escribir reglas, la máquina las **infiere a partir de datos**
- Tres elementos clave: datos de entrada, etiquetas y una función de pérdida
- Su historia no ha sido lineal: ciclos de entusiasmo y desencanto

Línea temporal

Ciclos de “verano” (avances, financiación) e “invierno” (desilusión, recortes de fondos).



Raíces matemáticas y el Test de Turing (1805-1950)

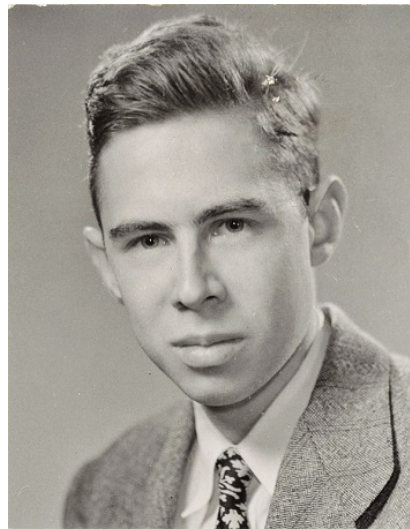
- 1805: Legendre y Gauss, **mínimos cuadrados**
- 1936: Fisher, análisis discriminante lineal
- 1943: McCulloch y Pitts, primera neurona artificial
- 1950: Alan Turing, "*Computing Machinery and Intelligence*" — el **Test de Turing**



Fuente: Elliott & Fry (1951), dominio público

El Perceptrón y los inviernos de la IA

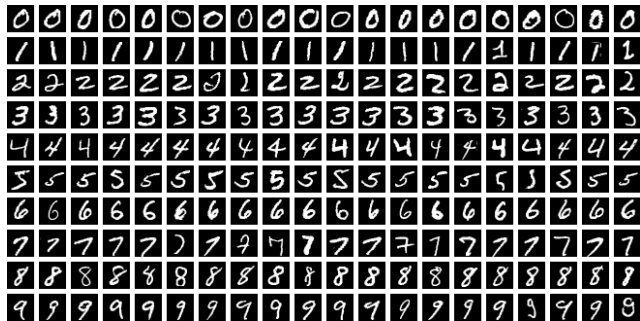
- 1958: Frank Rosenblatt crea el **Perceptrón**
- **Primer invierno** (años 70): el Perceptrón no resuelve problemas no lineales
- 1956: nace formalmente la IA (seminario de Dartmouth)
- **Segundo invierno** (años 90): sistemas expertos caros y poco escalables



Fuente: Heinz Nixdorf MuseumsForum,

Deep Learning y la era de los LLM (2010-hoy)

- 2012: victoria de Hinton en el desafío **ImageNet**
- **MNIST**: banco de pruebas clásico para clasificación de dígitos
- 2017: arquitectura **Transformer** — revolución en PLN
- 2020s: **IA Generativa** (ChatGPT, Gemini) a escala de miles de millones de parámetros



Fuente: Suvanjanprasai (2024), Wikimedia Commons, CC BY-SA 4.0

Datos, *features* y etiquetas

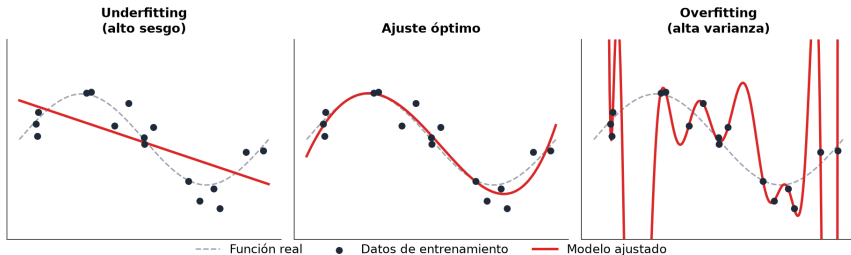
Muestra	Feature 1	Feature 2	Feature 3	Label
1	25	70	175	Gato
2	8	4	90	Gato
3	35	80	180	Perro

- **Muestra:** punto de datos individual
- **Features:** atributos que describen la muestra
- **Etiqueta (label):** lo que queremos predecir — la “verdad fundamental”

Overfitting vs. underfitting

- **Underfitting:** el modelo es demasiado simple, no capta el patrón (alto sesgo)
- **Overfitting:** el modelo memoriza el ruido de entrenamiento (alta varianza)
- Objetivo: el equilibrio sesgo-varianza que mejor generaliza

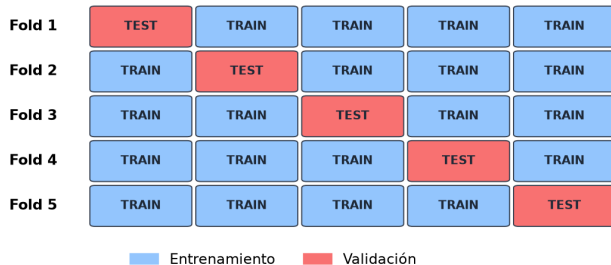
Underfitting vs. ajuste óptimo vs. overfitting



Validación cruzada K -fold

- Divide los datos en K particiones (*folds*); cada una actúa una vez como test
- Métrica final = media de los K resultados → estimación más robusta
- Evita depender de una única partición train/test

Validación cruzada 5-fold



Matriz de confusión y métricas

Matriz de confusión

		Predicción	
		Positivo	Negativo
Clase real	Positivo	TP Verdadero Positivo	FN Falso Negativo
	Negativo	FP Falso Positivo	TN Verdadero Negativo

$$\text{Precisión} = \frac{TP}{TP + FP}$$

$$\text{Recall} = \frac{TP}{TP + FN}$$

$$F1 = 2 \cdot \frac{P \times R}{P + R}$$

TP/TN: aciertos; FP/FN: errores. El F1 equilibra precisión y recall, útil con clases desbalanceadas.

El *pipeline* de ML: 10 etapas

- Proceso **iterativo**: la evaluación puede obligar a volver a etapas anteriores
- Gran parte del esfuerzo real está en las etapas 2-3 (datos) y 7-8 (ajuste)
- El despliegue no es el final: requiere monitorización continua



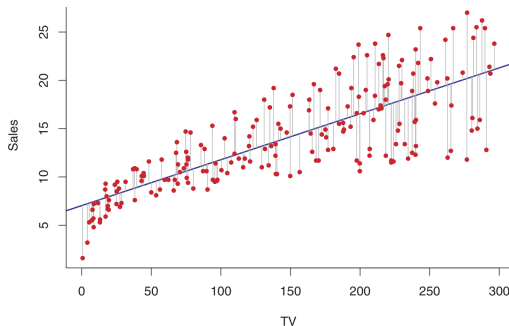
Sección 2

Aprendizaje Supervisado

- Predice un **valor continuo** Y a partir de predictores X
- Simple: $Y \approx \beta_0 + \beta_1 X$
- Múltiple: $Y = \beta_0 + \beta_1 X_1 + \cdots + \beta_p X_p + \epsilon$
- **OLS**: minimiza la suma de residuos al cuadrado (RSS)

Ejemplo real: dataset *Advertising*

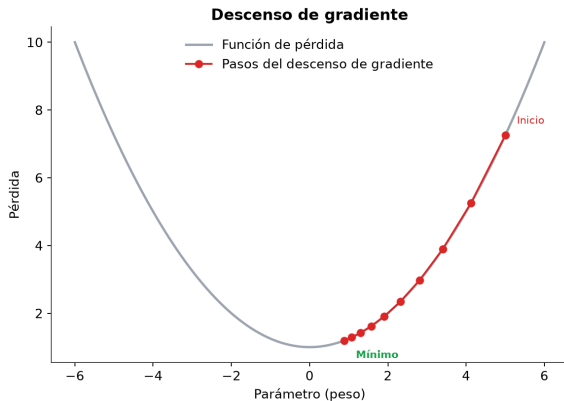
- Predice las ventas (*Sales*) a partir de la inversión en TV, radio y prensa
- La recta minimiza la suma de residuos al cuadrado (OLS)
- Ejemplo clásico de regresión lineal simple



Fuente: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 3.1. Libro de libre distribución (statlearning.com).

Descenso de gradiente

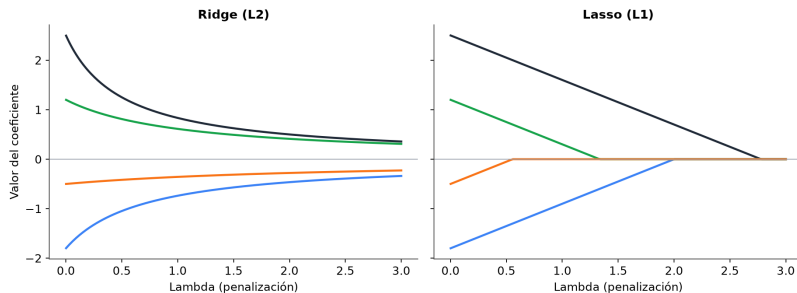
- Algoritmo iterativo: actualiza los parámetros en la dirección opuesta al gradiente
- $\theta \leftarrow \theta - \alpha \nabla J(\theta)$, donde α es la tasa de aprendizaje
- Una tasa muy alta diverge; una muy baja converge lentamente



Regularización: Ridge y Lasso

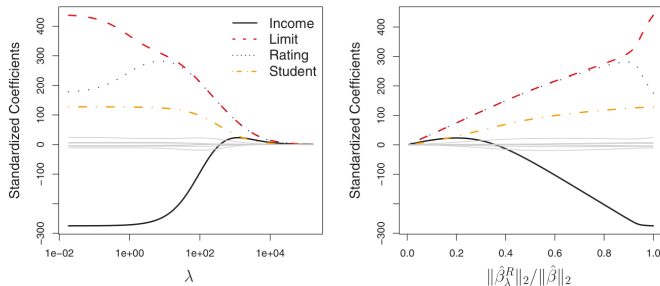
- **Ridge** (L_2): penaliza $\sum \beta_j^2$, encoge los coeficientes sin anularlos
- **Lasso** (L_1): penaliza $\sum |\beta_j|$, puede anular coeficientes (selección de variables)
- λ controla la fuerza de la penalización

Trayectorias de los coeficientes al aumentar la regularización



Ridge en datos reales: dataset *Credit*

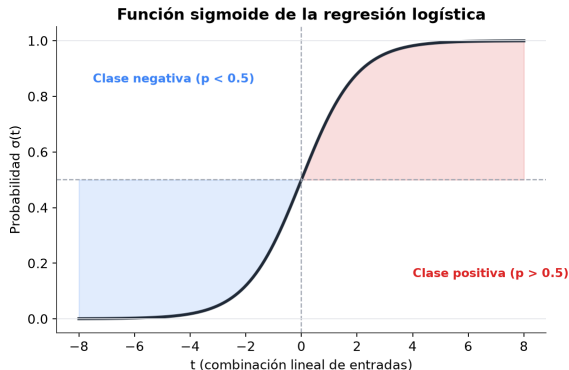
- Al aumentar λ , los coeficientes se encogen hacia cero
- Reduce la varianza a cambio de un pequeño aumento del sesgo



Fuente: James, Witten, Hastie & Tibshirani (2013), *An Introduction to Statistical Learning*, Fig. 6.4.

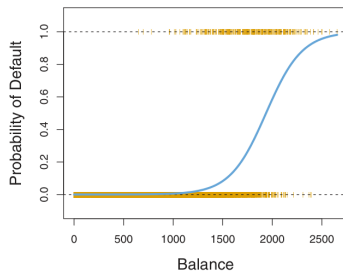
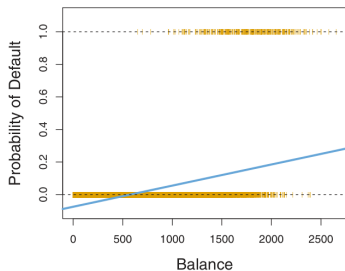
La función sigmoide

- Transforma cualquier valor real en una probabilidad entre 0 y 1
- $\sigma(z) = \frac{1}{1 + e^{-z}}$
- Es la base de la regresión logística



¿Por qué no usar regresión lineal?

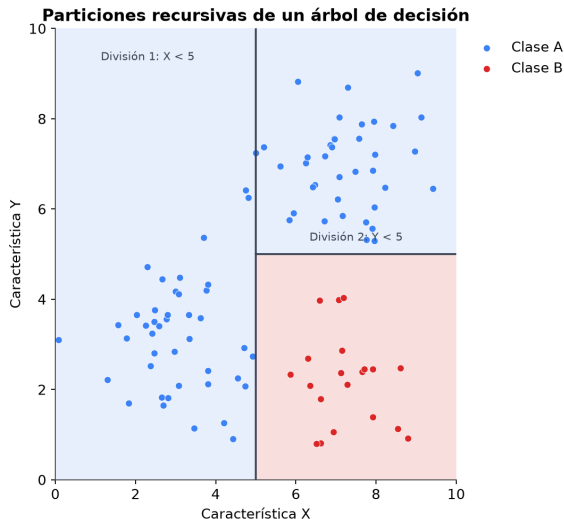
- La regresión lineal puede predecir probabilidades < 0 o > 1 (sin sentido)
- La regresión logística usa la sigmoide para mantener la salida en $[0, 1]$



Fuente: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 4.2.

Árboles de decisión

- Divide el espacio de *features* con cortes ortogonales sucesivos
- Cada división minimiza la impureza (Gini o entropía) de los nodos hijos
- Fáciles de interpretar, pero propensos a overfitting sin poda

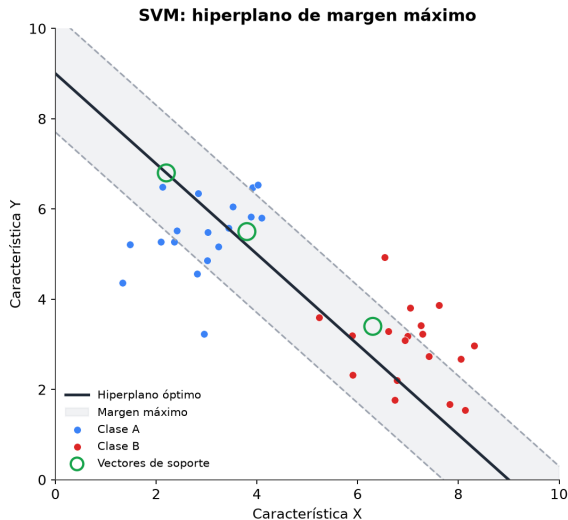


Ensembles: Random Forest y Boosting

- **Random Forest:** *bagging* + subconjunto aleatorio de features en cada división
- **Boosting:** modelos secuenciales que corrigen los errores del anterior (XGBoost, LightGBM)
- **Voting / Stacking:** combinan las salidas de varios modelos independientes

SVM: margen máximo

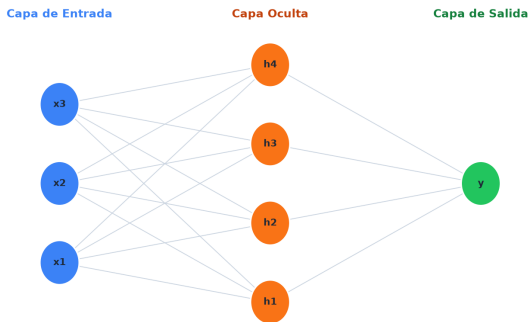
- Busca el hiperplano que maximiza el margen entre clases
- Los vectores de soporte son los puntos más cercanos al hiperplano
- El *kernel trick* permite fronteras no lineales



Redes neuronales

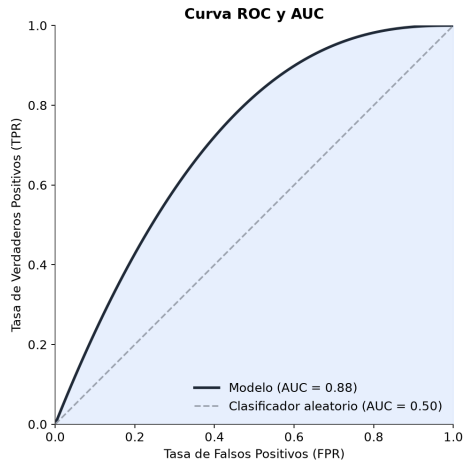
- Capas de neuronas conectadas por pesos ajustables
- Cada neurona aplica una función de activación no lineal (ReLU, sigmoide...)
- Entrenadas mediante retropropagación (*backpropagation*) del error

Arquitectura de una red neuronal totalmente conectada



Evaluación: curva ROC y AUC

- Eje Y: tasa de verdaderos positivos; eje X: tasa de falsos positivos
- $AUC = 1$: clasificador perfecto; $AUC = 0.5$: equivale al azar
- Compara modelos sin depender del umbral elegido



Sección 3

Aprendizaje No Supervisado

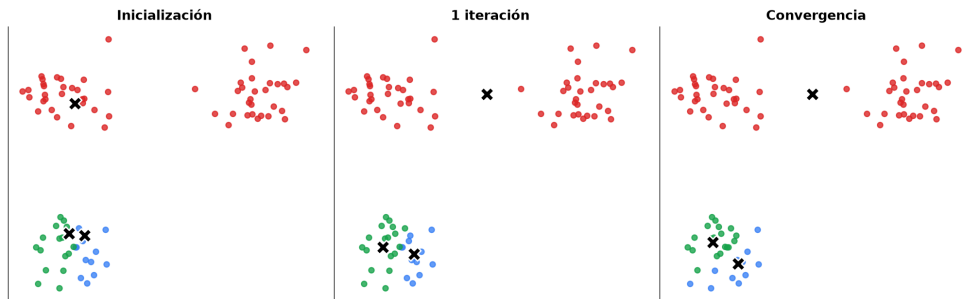
¿Qué es el clustering?

- Sin etiquetas: solo tenemos X , sin y
- Objetivo: agrupar datos con **alta similitud interna** y **alta diferenciación externa**
- Algoritmos: K -means, jerárquico, DBSCAN

Proceso iterativo de K -means

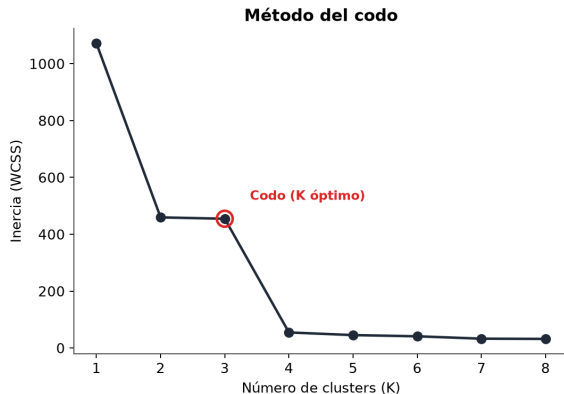
- 1 Inicializa K centroides al azar
- 2 Asigna cada punto al centroide más cercano
- 3 Recalcula centroides como la media de su cluster; repite hasta converger

Proceso iterativo de K-means



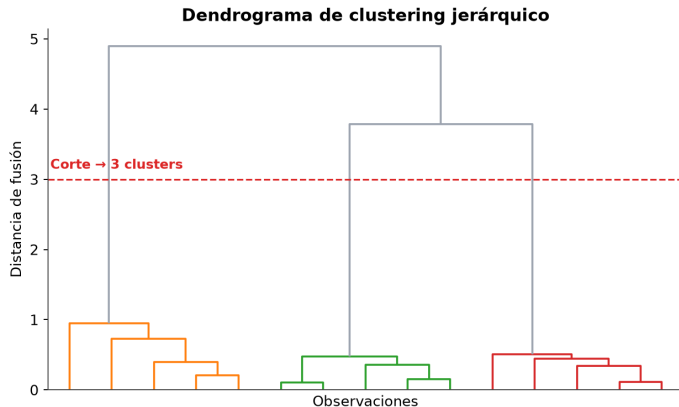
Método del codo

- Representa la inercia (suma de distancias al cuadrado) frente a K
- El “codo” señala el punto de rendimientos decrecientes: el K óptimo



Clustering jerárquico: dendrograma

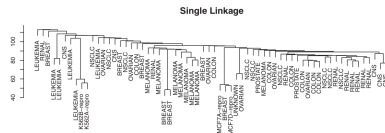
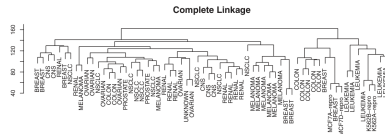
- Construye una jerarquía de clusters sin fijar K de antemano
- Cortar el dendrograma a una altura dada define el número de clusters



Datos reales: NCI60 (líneas celulares de cáncer)

- Agrupa líneas celulares de cáncer por similitud de expresión génica
- Líneas del mismo tipo de cáncer tienden a agruparse en la misma rama

Fuente: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 10.17.

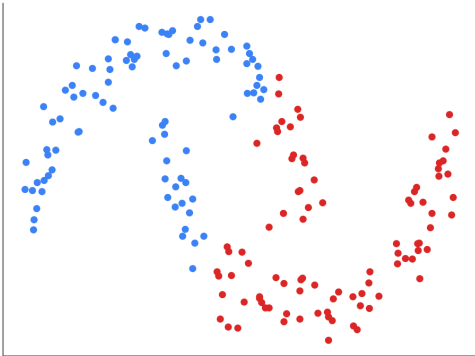


DBSCAN vs. K -means

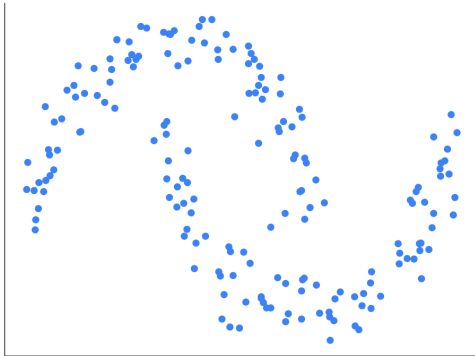
DBSCAN agrupa por densidad: no necesita fijar K , detecta formas arbitrarias e identifica automáticamente el ruido (puntos que no pertenecen a ningún cluster).

K-means vs. DBSCAN en datos con forma de luna

K-means (falla en formas no convexas)

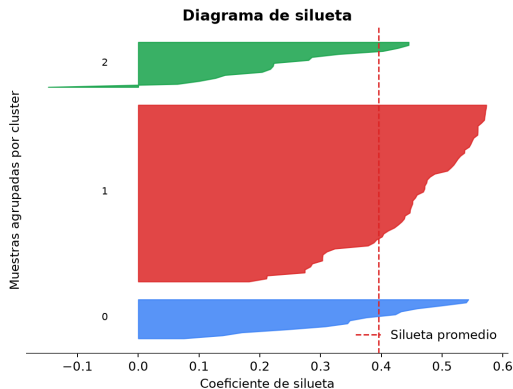


DBSCAN (detecta formas arbitrarias)



Evaluación: coeficiente de silueta

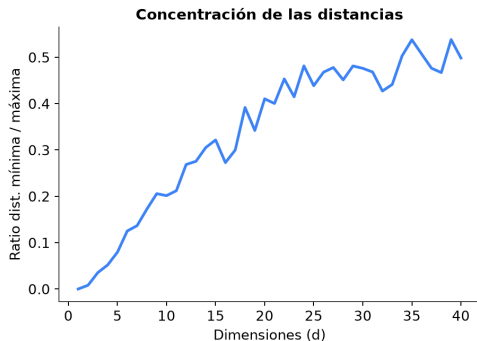
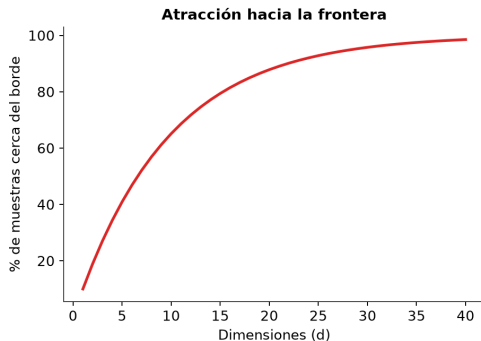
- Mide cuán similar es un punto a su propio cluster frente a los demás
- Rango $[-1, 1]$: valores altos indican clusters bien separados



La maldición de la dimensionalidad

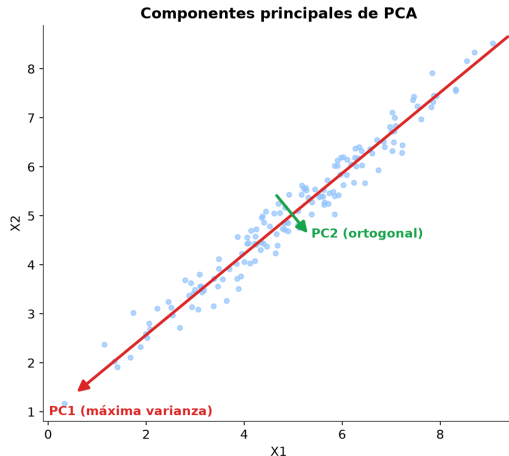
Al aumentar las dimensiones, los datos se vuelven extremadamente dispersos y las distancias entre puntos se vuelven casi indistinguibles.

La maldición de la dimensionalidad, cuantificada



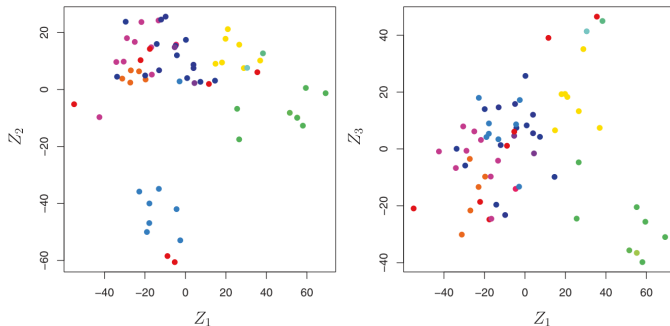
PCA: componentes principales

- PC1: dirección de máxima varianza; PC2: ortogonal a PC1, máxima varianza restante
- Reduce dimensiones conservando la mayor parte de la información



PCA en datos reales: NCI60 (6830 genes)

De 6830 genes a solo 3 componentes principales: las líneas del mismo tipo de cáncer (mismo color) se agrupan a pesar de la enorme dimensionalidad original.



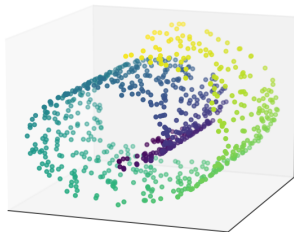
Fuente: James, Witten, Hastie & Tibshirani (2013), An Introduction to Statistical Learning, Fig. 10.15.

Manifold learning: el Swiss roll

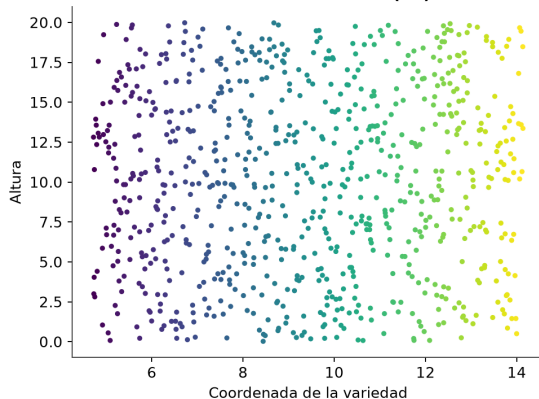
PCA (lineal) colapsaría puntos lejanos en la superficie enrollada; el *manifold learning* “desenrolla” la superficie preservando las distancias locales.

Swiss roll: motivación del manifold learning

Datos originales (3D)



Variedad desenrollada (2D)

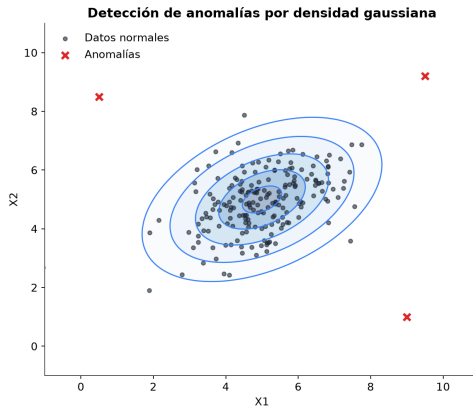


- **t-SNE**: preserva vecindades locales; excelente para visualización, lento en datasets grandes
- **UMAP**: más rápido, preserva mejor la estructura global
- Ambos son no lineales — capturan relaciones que PCA no puede

- *Inliers* (normales) vs. *outliers* (anomalías)
- **Anomaly detection**: dataset de entrenamiento contaminado
- **Novelty detection**: dataset de entrenamiento limpio

Enfoque gaussiano

- Ajusta una distribución gaussiana (multivariante) a los datos normales
- Los puntos con baja densidad de probabilidad se marcan como anomalías

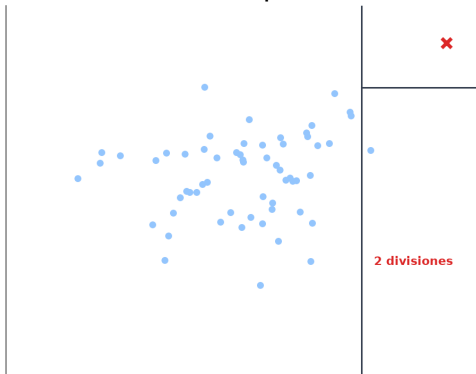


Isolation Forest

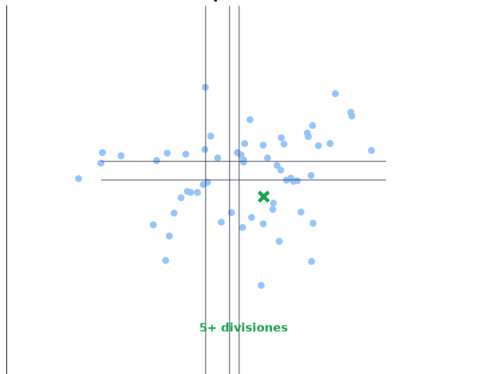
Aísla puntos mediante particiones aleatorias del espacio de *features*: las anomalías requieren muy pocas particiones para quedar aisladas.

Isolation Forest: intuición del aislamiento

Anomalía: aislada en pocas divisiones

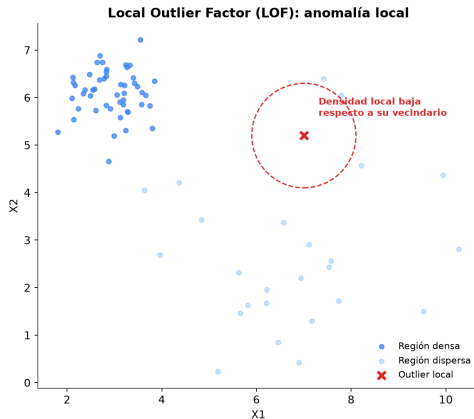


Punto normal: requiere muchas divisiones



Local Outlier Factor (LOF)

- Compara la densidad local de un punto con la de sus vecinos
- Detecta anomalías locales que un umbral de densidad global pasaría por alto



Fundamentos

- Historia del ML
- Overfitting/underfitting
- Validación cruzada

Supervisado

- Regresión, regularización
- Clasificación, SVM
- Redes neuronales

No supervisado

- Clustering
- PCA, manifold learning
- Detección de anomalías

¡Gracias!

Universidad de Salamanca